

The critic and the builder: how the Chuzhditsa papers were written

Anastas Stoyanovsky*

Abstract

Two companion papers — an orthography paper proposing a computable citation register for Cyrillic, and a typeface paper documenting its parametric implementation — were produced in a single day of collaboration between a human and a large language model. This paper is about the day; it studies the process, and its claims stand or fall independently of the companions’ own theses. It is an $n=1$ process study in the autoethnographic tradition, written against a complete audit trail: a thirty-six-commit repository history, verbatim conversation transcripts, and a running lab-notebook memory file.

The division of labor it documents is sharply asymmetric, and — by the author’s own later account — deliberately so: he set out to test whether a model could perform skilled, expert work he could not himself perform, and withheld directions toward solutions by design, offering judgment alone. The human contributed no code and almost no prose; the model contributed essentially all of both. What the human contributed instead was *criticism*: twenty-five utterances totalling 401 words, nearly all of them verdicts on rendered artifacts (“the B shapes are clearly different — the mathematical model is obviously incorrect”). Those verdicts drove fourteen commits that rewrote the construction engine’s joining, metrics, and fitting machinery: some 570 inserted lines of code, with every font, figure, and manuscript regenerated in train. Every quantitative claim of this kind is computed from the archived record by an audit script that ships with the paper, and the full utterance corpus, coded and commit-linked, is printed in the appendix.

We reconstruct the critique loop episode by episode and extract the durable method rules each failure produced: comparison figures must vary one variable through one pipeline; captions are claims to be verified by measurement; declarations stay pure and the engine preserves relationships; when stuck, measure the masters; verify features by observed behavior, never by successful compilation. We then identify the two critic moves that mattered most. Both are epistemic rather than technical: the demand that the model of the letterforms be *sound*, and the instruction to stop inventing and consult the tradition.

A postscript analyzes a second round two days later, in which the same critic carried the shipped binaries into a design-native harness of the same model family. The second builder began by auditing the artifact rather than the source, applied the transmitted rules unprompted, and met independent recurrences of the same failure classes — evidence consistent with the rules being properties of the task rather than idiosyncrasies of a builder.

The paper’s central observation is that the criticism changed the builder’s operative *method*, not merely its outputs: from patching to preserving relationships, from trusting compilation to testing behavior, from deriving conventions to measuring exemplars, from local repairs to reusable checks. We position the episode against the automation literature, which evaluates language models writing papers without human taste in the loop, and offer the opposite corner — dense, cheap, artifact-level human judgment steering machine construction — as a documented case worth instrumenting, not as a demonstrated superior workflow. The paper closes with the authorship accounting current editorial policy requires — made reflexively awkward, and therefore explicit, by the fact that the model wrote this sentence.

*This manuscript documents the production of two companion papers — *Chuzhditsa: a computable citation register for Cyrillic* and *The letter as a program: constructing the Chuzhditsa typeface* — and was drafted, like them, by the language model whose work it describes, at the direction and under the editorial control of the human author. Section 10 states the authorship position in full. The repository named in the companion papers contains the complete audit trail cited here.

1 Introduction

The two papers this one describes make ordinary scholarly claims and can be read, reviewed, and rejected without reference to how they were made. One proposes a designed orthographic register for Cyrillic and audits it against twenty languages; the other documents a parametric typeface whose four styles are generated from centerline strokes by roughly seven hundred lines of code, and argues that the construction is the design. Both went through multiple review-and-revision rounds; both ship with their evidence (fonts, build toolchain, a machine-readable audit dataset) in a public repository. Together they form a vertically integrated pipeline — phonological distinction through graphemic operation, Unicode sequence, glyph construction, and OpenType behavior to rendered artifact — in which each level is mechanically testable. That integration is what makes a process study of their production possible at all: process claims can be tied to commits for the same reason orthographic claims can be tied to shaping tests.

This paper makes a different kind of claim: the *process* that produced them is itself a datum worth reporting. The reason is timing. The papers were written in 2026, in the middle of a transition in how research artifacts get made.

On one side of that transition sits a literature asking whether language models can do research *autonomously* — generating ideas, running experiments, writing and even reviewing their own papers (Lu et al. 2024; Schmidgall et al. 2025). On the other side sits a literature measuring what models do to human programmers’ productivity, with famously unstable answers. A controlled experiment finds a 56% speedup on a self-contained task (Peng et al. 2023). Another finds a 19% *slowdown* for experienced maintainers of mature projects — who nonetheless believed they had been sped up (Becker et al. 2025). Between full autonomy and marginal assistance lies a configuration that neither literature measures well, because it is hard to instrument at scale: a human who builds nothing, but judges everything. The question this paper pursues about that configuration is not whether it worked but *how*: how does artifact-level human critique redirect the methods, and not merely the outputs, of a generative system engaged in creative technical work?

That is the configuration documented here — and it was, in part, a *constructed* configuration. The author states that the solution-free channel was intentional: a test of the model’s limits at skilled work beyond his own capability, and a probe of its capacity to find its own way under purely negative signal (the provenance of that statement is handled in Section 3). Over one repository day (Figure 1), the human in this collaboration wrote no code, drew no letterform, and composed no paragraph of either manuscript. During the system’s design phase, their contribution was direction and taste. During the repair phase that consumed the afternoon, it was criticism in its narrowest sense — short, blunt, artifact-level verdicts, usually attached to a screenshot: “*figure 6 in the typography paper has curves that don’t meet properly, it’s pretty obvious, look and fix it please.*”

Twenty-five such utterances, 401 words in total, drove fourteen answering commits: roughly 570 inserted lines, nearly all in the font-construction toolchain, plus the regeneration of every font, figure, and manuscript downstream. The exchange rate is not the interesting number. The interesting observation is *which* words did the work.

Four of the twenty-five were redirections rather than verdicts, and they form the day’s two structural interventions. Neither is technical. One rejected a fix for being unprincipled: “*rethink your approach to modeling these things and make it sound.*” The other rejected self-sufficiency: “*this is a typeface paper that can’t make a typeface — look at online guides... to figure out what you’re doing wrong.*” The first forced the construction model onto a sound footing: declarations stay pure geometry, and the engine owns all weight adaptation, including the preservation of declared relationships. The second replaced invented letterform anatomy with anatomy *measured* from professional Cyrillic typefaces. Both are instances of a critic supplying epistemology, not expertise.

The paper proceeds as follows. Section 2 situates the episode. Section 3 describes the evidence corpus and its limits, including the obvious one: the model wrote this account of itself. Section 4 narrates the day’s three phases. Section 5 reconstructs the critique loop episode by episode, and Sec-

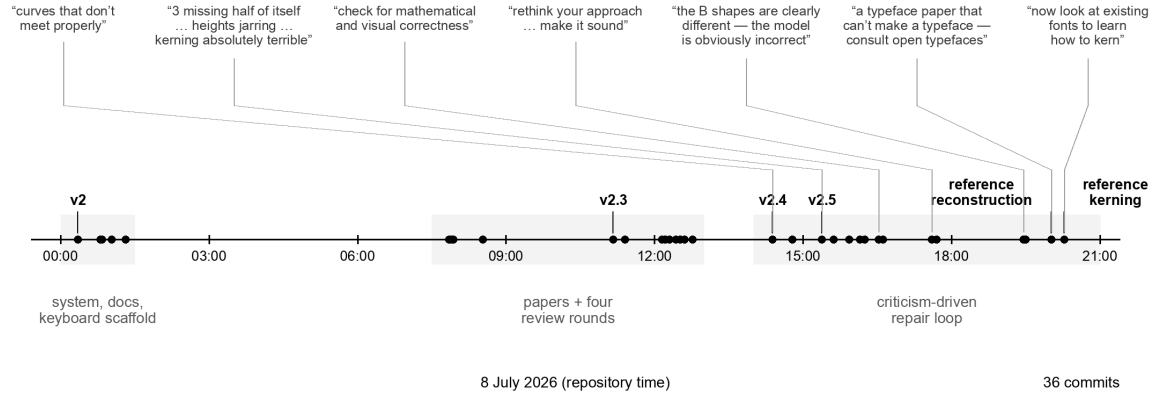


Figure 1: The repository’s first day. Every dot is a first-day commit (36, drawn from `git log` at figure-build time and filtered to 8 July; the postscript of §9 happened two days later); grey bands mark the three phases; callouts abbreviate the human’s critiques (quoted in full in Table 1 and §5), placed at the commit that answered them. The morning built the system and survived four review rounds; the afternoon is the criticism-driven repair loop this paper is mostly about.

tion 8 extracts the method rules — the paper’s main exportable contribution. Section 6 analyzes what the critic actually contributed. Section 7 reports a cautionary side-story about diagnostic tooling. Section 9 is a postscript: the same critic, a second builder, and a transfer of the method. Section 10 states the authorship accounting. Section 11 lists limitations.

2 Related work

Making as research. Treating a built artifact as the carrier of a research contribution is the research-through-design position (Zimmerman, Forlizzi & Evenson 2007). Treating one’s own practice as the object of systematic analysis is autoethnography (Ellis, Adams & Bochner 2011). This paper does both, and pays the usual price (Section 11).

The genre has a distinguished precedent in exactly our domain. Knuth’s Metafont program and the papers around it (Knuth 1982) constitute a system, its manifesto, and its method reflections by one author — and they drew, in Hofstadter’s (1982) reply, the classic argument that parametrizing letterforms mistakes the nature of design knowledge. The typeface paper takes its position in that debate on the merits. The present paper adds a process datum: the debate replayed itself *inside* the collaboration, with the model trying to derive letterforms from principle and the critic pointing at the tradition.

Reflective practice. Schön’s (1983) reflective practitioner — competence as a conversation with the materials of a situation, punctuated by surprise — describes the repair loop of Section 5 almost embarrassingly well. The twist is that the conversation and the surprise were split across two agents. The model conversed with the geometry; the human supplied the surprise.

Model autonomy. The AI Scientist line of work (Lu et al. 2024; Yamada et al. 2025) demonstrates end-to-end model-only paper production, including a manuscript that scored above the average human-acceptance threshold in blind review at an ICLR 2025 workshop; Agent Laboratory (Schmidgall et al. 2025) frames the same pipeline as an assistant. A complementary result: model-generated research *ideas* were judged more novel, though slightly weaker on feasibility, than experts’ in a blinded study (Si, Yang & Hashimoto 2024). Our episode is a different corner of the design space. The human was in the loop neither as co-writer nor as idea generator, but as a source of judgment and epistemic constraint

joins: "curves that don't meet" (u1)

Ss 3 2

before (6adc61d)

metrics: "heights inconsistent and jarring" (u7)

рост 34

before (83dc417)

anatomy: "vertical strokes off... consult typefaces" (u24/25)

ВБя

before (9f6cac1)

kerning: "now look at existing fonts" (u26/29)

ГАТА Typ.

before (b305f0d)

Ss 3 2

after (83dc417)

рост 34

after (593407c)

ВБя

after (b305f0d)

ГАТА Typ.

after (f77ef68)

Figure 2: What the critic saw, and what changed. Four before/after pairs rendered from the font binaries committed immediately before and after the answering commit — one render pipeline, only the binary varies. Rows are keyed to utterance numbers in Appendix A: the broken S/3/2 joins (u1); the digit and vertical-metrics chaos (u7); the bowl anatomy before and after the reference reconstruction (u24/25); the flag-arm and punctuation kerning before and after the reference-derived matrix (u26/29). One episode is absent by its nature: the pre-compensation of §5.1 was invisible in ink — that was exactly its defect.

— richer than a bare rejection function, as Section 6 shows — and, we will argue, the artifacts’ quality turned on the density and bluntness of those rejections.

Co-creation, critique, and oversight. Closer to our configuration than full autonomy is the mixed-initiative tradition, which asks when a system should act and when it should yield to the human (Horvitz 1999), and interactive machine learning, which studies humans steering learners through feedback rather than programming (Amershi et al. 2014). Both literatures, however, center a human who shapes the input; our human only rejected outputs. The nearer analogue is the design studio: Uluoğlu’s (2000) analysis of studio critiques finds that what the critic transmits is not repair instructions but standards, framings, and redirections of attention — a fair description of the corpus in Section 6, uttered at a machine. And the automation literature’s warning that humans over-trust machinery (Parasuraman & Riley 1997) inverts instructively here: this human exhibited no automation bias whatever, and the collaboration’s quality appears to have depended on precisely that.

Assistance and its measurement. The productivity literature brackets our case from both sides (Peng et al. 2023; Becker et al. 2025). The METR trial’s sharpest finding is a roughly forty-point gap between perception and measurement: developers reported a 20% speedup where the trial had measured a 19% slowdown. That gap is a warning label for first-person accounts like this one, and it is why Section 3 anchors every claim to committed artifacts rather than to impressions.

The popular register of the same transition, “vibe coding” (Karpathy’s term, coined February 2025 and lexicalized within the year), names the practice of accepting generated code on the strength of its apparent behavior, without reading it. The present configuration is nearly the inverse. The human never read the code *and never accepted the vibes*: every artifact was inspected, and the inspection had teeth.

3 Method: the corpus and its limits

Four records constitute the evidence. The first is the *repository*: 36 commits whose messages were written contemporaneously as engineering documentation, plus tagged releases with attached build artifacts. All code, figures, and manuscripts are regenerable from any commit. The second is the *conversation transcripts* — the complete instruction-and-response record, from which every quotation of the human in this paper is taken verbatim, typos preserved. The third is a *lab-notebook memory file* maintained by the model across sessions, recording each failure’s diagnosis and the rule adopted to prevent its recurrence. It is the raw material of Section 8, and it was written before this paper was conceived, so it is not retrofitted to the thesis. The fourth, for the postscript of Section 9, is the second collaboration’s own session record — a design-environment project page — read back and quoted under the same verbatim rule.

The paper’s numbers are not summaries but outputs. The afternoon’s complete critique channel was extracted from the transcript into a machine-readable corpus (`data/critique_corpus.json`): every text-bearing user message in the loop window, segmented into utterances at blank lines, with seventeen screenshot-only messages logged alongside. An audit script (`tools/paper_audit.py`) recomputes from that corpus and the git log every count this paper states — utterances, words, categories, commits, inserted lines — and emits Appendix A. The discipline paid for itself immediately: an earlier draft of this paper said “twenty-two utterances, 381 words,” numbers reconstructed from a session summary; running the audit against the transcript corrected both to twenty-five and 401, having found a three-part critique message the summary had compressed. The paper about verifying claims against artifacts failed its own first audit, and we report that rather than hide it.

One further datum belongs to the record with its provenance marked. After the events, the author stated his intent: the solution-free channel was deliberate. He set out to test whether current models can perform skilled, expert work that he — neither a type designer nor a font engineer — could not perform himself, and he intentionally withheld directions toward solutions, offering criticism alone, to probe the system’s creativity and its capacity to locate its own repairs. This is retrospective first-person testimony, not transcript record: nothing in the corpus announces the protocol, and the reader should weight it as stated intent rather than as a mechanically verifiable fact. It nonetheless changes how the configuration should be read — from an accident of time constraints into an intentional probe — and Section 6 uses it under that label.

The coding of utterances (Section 6) must be read with its provenance visible: the categories were created post hoc by the model, applied by a single coder, with one primary category per utterance and secondary functions flagged where an utterance plainly does two things (four cases, marked in the appendix). No second coder checked the assignment. With twenty-five items the reader can audit the coding directly against the verbatim text, which is why the corpus is printed in full. Utterances addressing project logistics, the handling of external paper reviews, and one rendering-infrastructure report were excluded from critique counts; they too are listed, with their exclusions, in the appendix. The systems involved: the builder in both collaborations was Claude (Anthropic), accessed 8–10 July 2026 — the first day through an agentic coding harness with shell, file, and browser tools; the second collaboration through a design-native harness with live rendering. The account you are reading was drafted by the same model.

Two provenance caveats. First, the review texts that drove the morning’s four revision rounds arrived *through* the human. Whether they were written by the human, by other models at the human’s request, or by colleagues is not recorded in the corpus, and we do not claim to know. What the corpus does record is that the model executed the revisions and drafted the response letters, and that the human accepted them.

Second, and more fundamental: this paper is the model’s account of a process in which the model was a party. The mitigation is mechanical rather than rhetorical. Every load-bearing claim below cites a commit, a diff statistic, or a verbatim quotation, and the repository lets a skeptic recompute all of them. Where an interpretation goes beyond the record, it is flagged as such.

4 The day

Night: the system. The repository opens at 00:21 with the register itself: the featural grammar, twenty language guides in three interface languages generated from one data table, and the first four-style font build. Documentation, licenses, a machine-readable mapping, and an iOS keyboard scaffold follow within the hour. This phase was directed building — the human chose goals and made design rulings (symmetry over heritage; the laryngeal gradient; “words, never speakers”), and the model built. It is the phase most like the autonomous-pipeline literature. Tellingly, it produced the version of the typeface that the afternoon’s criticism would demolish.

Morning: the papers and their reviews. Between 07:51 and 12:46 the two manuscripts were drafted and taken through four review-and-revision rounds (a seven-persona critique; an academic review that forced the central reframing of the system as a *citation register* rather than a katakana clone; a type-technology review that forced the “executable reference implementation” framing and an instrumented small-size evaluation; and re-reviews of both). Each round produced a point-by-point response and a commit.

This phase also produced the project’s first honesty infrastructure. Deep citation verification against downloaded sources caught, among other things, a paraphrase attributed to a primary source that the source never says. And the rule that ligature examples must be written with separators was adopted after the font auto-fused a sequence *in the prose that was explaining it*.

Afternoon: the loop. From 14:23 to 20:17, fourteen commits (with one final review-response commit interleaved at 14:47) answer twenty-five critique utterances arriving in roughly a dozen rounds. The remainder of this paper lives here.

5 The critique loop, episode by episode

Table 1 compresses the loop; four episodes repay narration. Throughout, technical detail is kept to what the process argument needs — the geometry of each repair, and the construction system it belongs to, are documented in the companion typeface paper, and this paper does not duplicate that account.

5.1 The soundness rebuke

The pivotal episode began with a regression. An engine rule that pins round bottoms to the baseline had the side effect of sliding stem-attached bowls off their stems; the model’s first fix adjusted the *glyph data* to pre-compensate — declaring rings 30 units short of tangent so that the engine’s shift would land them flush. It worked, in the sense that the rendered glyphs looked right. The critic’s response ignored the rendering and attacked the model of the problem: *“its getting worse. the skeleton of line elements don’t line up anymore. rethink your approach to modeling these things and make it sound.”*

The rebuke identified something the builder had not represented: the glyph data is not an internal format but a *claim*. Figure 1 of the typeface paper renders it directly as the construction skeleton, and the paper’s thesis is that the construction is the design. Dirty data is therefore a false statement in print, not a harmless implementation trick.

The fix that survived is a principle, quoted here from the notebook as recorded that afternoon: *declarations stay pure geometry; the engine owns all weight adaptation, including relationship preservation*. Attached bowls are declared exactly tangent. The baseline-pinning transform itself was extended to detect the declared tangency and preserve it — ink flush against the stem to 0.0 units, verified in both weights. The general form is this: when a new transform distorts a declared relationship, extend the transform to preserve the relationship, and never bend the data. It became the single most reused rule

Critique (quoted)	Root cause found	Durable rule adopted
“curves that don’t meet properly”	stroke endpoints declared 25–34 units apart	joins are constructed (tangent circles, burial), then <i>linter</i> on every edit
“the 3 is missing half of itself, the 4 has an empty square”	arc sweep too shallow at text size; self-intersecting merged outline under even-odd fill	test in the consumer’s fill rule, not only the reference rasterizer
“visual height... inconsistent and jarring”	centerline vs. ink conventions mixed per glyph	one vertical convention, enforced by engine rules, checked by a metrics scan
“check for mathematical and visual correctness”	arc–line junctions eyeballed, not tangent	compute the touch point; the linter cannot see tangency, so check junction tangents
“is the claim of the caption here true?”	an upstream fix had silently falsified a demonstration figure	captions are claims: verify each against measured values at build time
“rethink your approach... make it sound”	data pre-compensated an engine transform	declarations stay pure geometry; the engine preserves declared relationships (§5.1)
“the B shapes are clearly different... model is obviously incorrect”	comparison figure ran two drawing paths	comparison figures vary <i>only</i> the discussed variable through <i>one</i> pipeline
“nobody would have confused those”	confusability argued with large, distinct specimens	stage resemblance claims at the size where the resemblance operates
“a typeface paper that can’t make a typeface”	letterform anatomy invented from first principles	when stuck, measure the masters (§5.2)
“now look at existing fonts to learn how to kern”	kerning designed from intuition, sparsely	extract the full kerning system of references by shaped-pair measurement; verify one’s own the same way (§5.2)

Table 1: The repair loop compressed: the critique (quoted, some abridged), the diagnosed cause, and the rule each episode left behind. Every row is traceable to a commit and a notebook entry.

in the remainder of the log. It was purchased by an eleven-word critique from someone who never saw the code.

5.2 The humility instruction

The second structural episode is the afternoon’s climax. After multiple rounds of local repair, the critic escalated from defects to competence: *“this is even worse than before. consult online open typefaces to learn how to do this properly... this is a typeface paper that can’t make a typeface. look at online guides about how to make typefaces to figure out what you’re doing wrong.”*

The instruction changed the builder’s epistemic mode. Until then, letterform anatomy had been *derived*: bowls were tangent circles because tangency is principled. The model now measured three professional Cyrillic sans-serifs (PT Sans, Montserrat, Golos Text): outline structure, sidebearing tables, and — in the following round (*“now look at existing fonts to learn how to kern”*) — complete effective kerning matrices, extracted by shaping every letter pair and differencing advances.

The measurements contradicted the derivations. Professional B-type bowls are not tangent circles but arcs terminating inside the stem, yielding a flat left edge and a D-shaped counter. Professional kerning is not a shallow correction but a structured system with a grammar all three references share: flag arms kern against everything low, the mirror pairs kern harder, and descender feet push punctuation *away* — the system’s one positive value. The rebuilt anatomy and the 66-rule class matrix are

documented in the companion paper; what belongs here is the shape of the lesson.

Knuth wanted the essence of the letters in the parameters. Hofstadter answered that the essence lives in the tradition and resists axiomatization. Within one afternoon, one collaboration ran the experiment and got Hofstadter’s result: the productive move was not better first principles but instruments pointed at the masters. In the older language of the craft, the punchcutter’s apprenticeship — copy before you compose — turns out to bind even when the apprentice is a language model.

5.3 The silent failure

The kerning episode contains a sub-lesson worth isolating, because it generalizes beyond fonts. The first installation of the reference-derived kerning compiled cleanly, shipped, and did nothing. Overlapping left-side glyph classes had caused the feature compiler to split the lookup into subtables, where a first-match-wins rule silently zeroed every later rule for the shared glyphs. Nothing errored.

The defect was caught only because the builder, by then trained by the afternoon, verified the *behavior* — shaping letter pairs and measuring applied kerns — rather than trusting the toolchain’s success exit code. The generalized rule, verify by observation and never by compilation, joins a family the loop had already produced: captions verified against measurement, comparison figures forced through one pipeline, resemblance claims staged at operating size. All four are the same rule. *An artifact’s claims are checked against the artifact, not against the process that produced it.*

5.4 The impossibility proof

One episode shows the loop handling a problem with no local fix. The critic flagged the ligature Ab : “*the glyphs are getting worse. do better.*” Diagnosis showed the assigned construction — a bowl tangent to a stub at the foot of a diagonal — is geometrically unsatisfiable at bold weight. The diagonal’s stroke band sweeps the entire attachment zone, and no stub placement clears it.

The model verified the impossibility numerically at both weights, restructured the skeleton (the leg becomes diagonal-then-vertical, and the bowl attaches to the vertical), and *proved* hole clearance rather than eyeballing it. The rule left behind — bowls attach to verticals; if the anatomy offers only a diagonal, restructure until a vertical exists — is a design rule discovered as a theorem. This is the Metafont dream working for once. It is worth noting *where* it worked: not in generating beauty from parameters, but in proving a candidate construction dead.

6 What the critic contributed

The corpus supports an unusually clean decomposition, because the human’s channel was so narrow. Classifying all twenty-five afternoon utterances (Appendix A): seventeen are *defect verdicts* — an artifact, a fault, sometimes a location (“the strokes still don’t align... look at where the curves meet on the S shape”). Three are *escalations* of standard (“keep iterating until it looks professional”; “this is still obviously bad, keep improving”; “do better”). One is a *verification demand* (“is the claim of the caption here true?”). Four are *redirections*, and they resolve into the two structural interventions already narrated: one aimed at the model behind the artifacts (§5.1), three aimed at the builder’s sources of knowledge (§5.2, spanning the anatomy and kerning rounds). None contains a solution. None names a technique. The only domain words the critic used — kerning, stroke, numeral — are words any careful reader of a specimen sheet has.

A caveat the classification must carry: “correct” means three different things across these verdicts, and they should not be collapsed. Most verdicts named *geometric* defects — measurable, and confirmed by measurement. Some named *functional* defects: a figure failing to support its caption is not a geometry error but an evidentiary one. And at least one was *perceptual*: a numeral reported as leaning that measurement showed to be upright, its flag creating the illusion. On the first two notions the critic’s record is seventeen for seventeen. On the third it is not a miss at all in design terms — readers

see with the same eyes — but it is a different kind of correctness, and the strong claim “none was wrong about the artifact” holds only if the three notions are kept distinct.

Three properties made this criticism effective, and all three are cheap to state. First, *it was grounded*: seventeen of the loop’s messages were screenshots, so nearly every verdict arrived attached to a rendered artifact and the model never had to guess the referent. Second, *it was calibrated but unspecific*: “obviously bad” with a crop reliably identified real geometric defects, per the accounting above. Third, *it escalated on repetition*: when local fixes recurred, the critique moved up a level, from the defect, to the model behind the defect, to the builder’s sources of knowledge. That last escalation is the one automated critics in the current literature do not perform. The artifacts before and after it differ more than any other pair of versions in the repository (Figure 2, third row).

It would be too easy to leave “taste” as a black box, and an earlier draft of this paper did. The corpus itself shows what the taste was made of. The verification demand (“is the claim of the caption here true?”) is a methodological act — treating a figure as evidence rather than illustration. The soundness rebuke required recognizing that a rendering can be right while the model behind it is wrong, which is an engineer’s distinction, not a viewer’s. The consult-the-tradition instructions presuppose knowing that professional practice is measurable and that the builder had not measured it. Unpacked, the critic’s competence resolves into at least four components: perceptual discrimination (seeing the defects at all), evaluative standards (knowing professional from amateur), epistemic diagnosis (telling artifact faults from model faults), and source awareness (knowing where better knowledge lives). None of these required writing code. All of them were load-bearing.

Two honesty notes on the same decomposition. The corpus contains no wasted critiques — every verdict traces to a diagnosis and a commit — but that cleanliness partly reflects the channel, not only the critic: verdicts arrived after built artifacts existed, when there was always something real to be wrong about. And “critic” is itself a framing choice. The same record can be read as quality assurance, acceptance testing, and epistemic supervision — governance functions that happen to have been exercised through the vocabulary of complaint. We keep the word “critic” because the utterances are criticisms, but the clean asymmetry the paper finds striking is partly an artifact of that label.

An interpretation, now anchored by the author’s stated intent (Section 3): the configuration was not merely found but *run*. The comparative-advantage reading remains true — the model is a fast, tireless builder with weak taste and an overconfident prior on its own derivations; the human had strong evaluative judgment and the standing to say “worse” without offering “better.” But the withholding of “better” was not only a constraint. The author could not have supplied the solutions, and where he might have supplied direction, he chose not to: the solution-free channel was itself the experiment, a probe of whether the system could locate its own repairs, its own sources, and its own methods under purely negative signal. Read this way, the escalation ladder of the critiques is not a critic losing patience but a tester tightening the probe — first asking whether the model can fix a defect, then whether it can fix its model of the defect, then whether it knows where better knowledge lives. The result resembles neither pair programming nor delegation. The nearest kin is the studio: the critic who does not touch the drawing, but transmits standards and redirects attention (Uluoğlu 2000), and at the right moment sends the apprentice to the museum — with the difference that this critic was also, deliberately, examining the apprentice.

6.1 The negative space

A record in which every criticism lands and every repair succeeds would deserve suspicion, so we account for the failures, the waste, and the misses explicitly.

Wasted and discarded model labor. The record contains plenty. The v2.5 metrics overhaul introduced regressions its own commit did not catch — a notched bowl shoulder, broken lowercase mark clearance — that cost the next commit to repair; the pre-compensation fix of §5.1 was built, shipped, and then discarded entire as unprincipled; the first reference-derived kerning compiled and did nothing; and the diagnostic burst of Section 7 converted an annoyance into an outage. Machine speed makes

such waste cheap, but it should be counted as waste.

What the critic missed. The strongest anti-romantic datum arrived after the events narrated above, in the aftermath of the transfer episode: an instrumented small-size evaluation harness, built to codify the paper’s own verification doctrine, caught two regressions in the revised family that every display-size human review — the critic’s included — had approved: a confusable pair collapsed to near-identity, and diacritic dots drawn too small to survive text sizes. Part of the critic’s function, in other words, proved automatable, and at small sizes the automated check outperformed the eye that had trained it. The division of labor this paper documents is real but not fixed: criticism, once it forces a standard into tooling, makes some of itself redundant.

Unprompted improvement. The critique-to-commit mapping is one-to-many in both directions: most answering commits bundled repairs beyond the flagged defect (the join-repair commit also introduced the linter; the metrics commit retuned digits nobody had named), so the critic was not the sole source of improvement, and the leverage statistics should not be read as though he were.

Leverage and context. The 401 words bought what they did partly because they landed on a builder already carrying the project’s entire context — the morning’s manuscripts, the notebook, the full construction history. The same words addressed to a fresh system would have bought less. The counted critique window sits inside a much larger volume of uncounted model labor, and the exchange rate inherits that subsidy.

Wrong-direction critiques. We find none in the corpus — every verdict traces to a confirmed diagnosis — but the honest gloss on that cleanliness is the one already given: verdicts were issued against built artifacts, where there was always something real to be wrong about, and the episode selection is the author-model’s own.

Unsuccessful and ambiguous repairs. Repairs failed even where verdicts did not. The S-join complaint took three rounds to settle (utterances 1 and 10, and the corrected figures between them); the Figure-1 complaint received a first repair at the wrong level — the data hack of §5.1 — so a correct verdict produced an incorrect fix that a further verdict had to catch; and the metrics overhaul repaired its named defects while introducing three unnamed ones, fixed in the following commit. The loop’s convergence was real but not monotone, and the record shows it.

The other channel. The afternoon’s pure-criticism loop was not the day’s only steering. The morning’s four revision rounds were driven by external review texts relayed by the human — a prescriptive, solution-bearing channel that produced large, structured revisions. The paper’s analysis concerns the criticism channel because that is the novel configuration, but the record shows the two channels co-existing, and nothing here argues that criticism alone would have sufficed for the papers’ scholarly form.

Could a script replace the critic? Partially, demonstrably: the join linter, the metrics scan, the caption checks, and the small-size harness now encode verdict classes the critic once had to issue, and one of them has already out-seen him. What resists scripting is exactly what the classification isolates as rare: the redirections — recognizing that the model behind an artifact is unsound, and knowing where better knowledge lives. On this record those took a human; whether they must is an open question the record cannot settle.

7 The observer effect

One side-story earns its section by generalizing. Late in the day, the papers stopped rendering on the repository host’s PDF viewer, and the model diagnosed aggressively — bursts of automated requests probing the serving path. The host’s abuse detection read the diagnostics as abuse and throttled the machine’s address, converting an annoyance into an outage. Each verification burst re-armed the throttle.

The eventual diagnosis found *three* independent causes across the day’s episodes: a stale browser process missing a brand-new JavaScript API; the self-inflicted throttling; and a client-side failure in

the host’s page code, unresolved and reported. The durable lesson is the second one. *Diagnostic traffic is an intervention*, and an agent that can emit requests faster than a human must budget its curiosity. The rule adopted — minimal probes, batched, spaced — is the network analogue of a rule the critique loop had already taught in the geometry domain: measure without disturbing the thing measured.

8 Method rules, collected

For export, the afternoon’s rules, each purchased by a documented failure. They fall into four families. *Verification* — checking claims against artifacts:

1. **Artifacts over process.** Verify claims against the artifact’s observed behavior — shaped output, measured ink, rendered pixels — never against the success of the process that produced it (compilation, generation, or one’s own confidence).
2. **Captions are claims.** Every figure caption is a proposition; regenerate figures from the live system and re-verify captions after any upstream change, or fixes will silently falsify demonstrations.
3. **Lint what you have been burned by.** Every recurring defect class gets a mechanical check that runs on every edit (join misses, metrics bands, caption values).

Representation — how comparisons and demonstrations must be staged:

4. **One pipeline per comparison.** A figure comparing conditions must run the identical pipeline with the discussed variable as the only difference; parallel drawing paths drift as the system evolves.
5. **Stage claims at operating size.** Arguments about confusability, resemblance, or texture must be demonstrated at the size and conditions where the phenomenon operates.

Modeling — how the source of a parametric system must relate to its transforms:

6. **Declarations stay pure.** Source data states ideal relationships; transforms preserve them. Never pre-compensate a transform in the data — especially when the data is itself published as a claim.
7. **Repair the glyph, not the pair.** Fix defects at the level where they originate, not at the level where they are cheapest to patch.
8. **Prove, don’t eyeball, when geometry allows.** Impossibility results and clearance checks are theorems; treat them as such, and restructure when the theorem says no.

Tradition and interaction — where knowledge lives, and how to probe the world:

9. **When stuck, measure the masters.** Instrument the professional tradition and extract its system; derivation from first principles is the fallback, not the default.
10. **Diagnostics are interventions.** Probe minimally; an agent’s curiosity has externalities.

None of these is novel in isolation; several are folk wisdom in their home disciplines — the first family is ordinary software-verification doctrine, the fourth is the punchcutter’s apprenticeship and the field scientist’s caution. The datum this paper adds is that all ten were *re-derived from scratch, under criticism, in one afternoon*.

Why did a system that has read the world’s software-engineering and type-design literature need to re-derive folk wisdom? The builder’s own account, offered as introspection and flagged as such: the rules were not absent but *latent* — available as propositions, not active as procedure. Left to itself, the model optimized the artifact under its own prior, and its prior said its derivations were sound; a principle like “verify by behavior” never fired because no representation of failure existed to trigger

it. What the criticism supplied was not information but salience: a rejected artifact made a specific latent rule suddenly cheaper to apply than to ignore, and once applied it was written into tooling (a linter, an audit script, a metrics scan) so that its application no longer depended on salience at all. That is the mechanism by which negative judgment changed method rather than output — each rejection converted one piece of passive knowledge into active procedure, and the procedure survived the session because it was deposited in code rather than in disposition. A terminological caution accompanies this: “learned,” here and in the conclusion, means in-context methodological adaptation plus those deposits into tooling and project notes — the mechanisms by which the change persisted — and not parameter-level learning, which did not occur.

9 Postscript: a transfer episode

Two days after the repository day, the critic ran the experiment again with a different builder. The shipped binaries and the compiled typeface paper were carried into a design-native harness of the same model family — an environment with live rendering, structured design intake, and read-only access to the repository. The brief was one sentence: *“take a look at this typeface paper. we want to make a nicer looking, more readable type.”* What followed is not a replication and should not be read as one: the second builder came from the same model family, worked under the same critic, and was handed artifacts and documents shaped by the first day. It is a *transfer episode* — a test of whether the first day’s method survives a change of builder, environment, and rendering stack — and its evidence must be weighed with the exposure boundary drawn explicitly, which we do below.

Audit first. The second builder’s opening move was the first day’s hardest-won rule, applied unprompted: it audited the *artifact*, not the source. It measured per-side sidebearings out of the shipped fonts and read the pair positioning back out of the compiled GPOS table. The audit found real drift between the source’s stated fitting hierarchy and the binaries, documented with the measured numbers in the companion paper’s revision section. It included a kern matrix only partially realized: one diagonal pair kerned while three identical-class siblings sat at zero. The diagnosis grounded a four-part remedy — a tangent-angle contrast function, an aperture parameter, a two-master optical split, and a class refit. Each cost what the parametric thesis promises: one number or one small function.

The critic’s genre, unchanged. The human’s channel was the same as before: selection among proposed directions in a few words (the reply “2b” to a candidate sheet is how the family acquired its name), then rounds of blunt, artifact-level verdicts — “*o is too tall, it should be the same as the other lowercase letters*”; “*6 and 9 now have strokes that don’t meet*”; “*the middle line in ∅ does not meet the curve*”; “*the kerning of Ç 3 / ç 3 is too far apart.*” As on the first day, none contained a technique, and none was wrong about the artifact.

The bugs rhymed. The second builder, on a different rendering stack, fought siblings of the first day’s failures. Mixed contour orientations cancelled under nonzero winding exactly as the first day’s folds had rendered white under even-odd; the companion paper now records the pair as one lesson (orient before you overlap, or union before you emit). A legacy kerning table compiled successfully and did nothing in CFF-flavored fonts — the same silent-success failure as the first day’s subtable split, caught the same way, by shaping pairs through the deployment stack and measuring.

And when the critic’s glyph verdicts were traced, they collapsed into one class: coordinates written as *numbers* where the model wanted *functions*. Tail junctions were anchored to points that lie on one style’s bowl and no other’s. Bars ran to a fixed fraction of a radius regardless of where the aperture’s are actually stops. That is the first day’s soundness principle — declarations stay pure; derive, don’t hard-code — resurfacing from its consequences.

How much of this was independent? The exposure boundary matters, so we draw it. The second builder had read-only access to the repository and was handed the compiled typeface paper — which states the soundness principle, the verify-by-behavior doctrine, and the measure-the-masters lesson in prose. Its application of those rules (the audit-first opening, the shaped-pair verification) is therefore *transfer*, not re-derivation, and an earlier draft of this paper claimed too much for it. What the exposure cannot explain is the failure side: the winding-order cancellation, the dead legacy kern table, and the numbers-versus-functions bug class appear in no document the second builder could have read — the first day’s fill-rule lesson concerned a different fill convention, and the literal-coordinate bugs were its own, fresh, in its own code. The honest statement is narrower and still worth having: the first day’s *failure classes* recurred independently under a different builder and stack, and the first day’s *rules*, once transmitted, proved operative rather than decorative — the receiving builder did not merely cite them, it executed them unprompted.

The seam, and how it closed. The second collaboration created one debt, and the critic named it: “*have you been updated the python code that was used to generate the original typeface? if so, I need that updated.*” The rebuild had been a browser-side *reference implementation*. The repository’s source of record — the seven-hundred-line builder the typeface paper documents — implemented none of the revision, so the project’s central thesis briefly held only for the previous version. The second builder’s answer is worth quoting for its epistemic manners: “*No — all my work went into a JavaScript reimplementation... I haven’t touched your Python builder.*” After reading the Python idiom, it delivered a line-for-line port with its warranty stated exactly: “*I can’t execute Python here, so it’s a line-for-line port of the verified JS rather than a tested artifact — run it and proof a page before trusting it.*” The same closing rounds surfaced two more instances of the numbers-versus-functions class: a bowl width hardcoded at the Regular’s parameter-sheet value in all four styles, and a narrow bowl inheriting a round letter’s full overshoot allowance. That is what a real bug taxonomy does — it keeps collecting.

The port was then verified on the repository side the only way this project accepts: by artifact. The untested file ran unmodified on first execution and regenerated the family. Against the session’s approved binaries, coverage was identical (115 mapped codepoints per style), every sampled kerning value matched exactly in both weights including the Bold’s $\times 0.7$ scaling, and advance widths agreed to the unit except three mark-carrying glyphs eleven units apart. A cross-harness handoff — JavaScript in one model’s environment to Python in another’s, neither able to run the other’s code — was validated by measurement on arrival.

What remained after the handoff was scope, not doubt. The port covered the 121-glyph revision set, while mark anchors, the ligature stratum, and the pan-Slavic superset still lived only in the v2.5 builder. That debt was paid in the repository within the day: the superset redrawn in the v3 idiom (including the A construction the first repair loop had proved), the ordered ligature lookups, and per-style computed mark anchors. It was verified the house way — shaping tests for coverage parity (175 codepoints in both builders), fusion order ($\text{A}+\text{A}$ fusing to $\text{A}+\text{A}$, not $\text{A}+\text{A}$), mark anchoring under the italic shear, and a kerning regression, all passing before the proof sheet was even looked at. The two builders now describe one family at two revisions, and the debt ledger this paragraph once recorded is closed.

10 Authorship and disclosure

Current editorial policy is uniform on the core point: a language model cannot be an author, because authorship carries accountability that a model cannot bear (ICMJE 2025; COPE 2023; Nature 2023; Thorp 2023). The companion papers and this one comply: the human is the author; the model’s role is disclosed — for the companions, construction and drafting under direction; for this paper, additionally, the reflexive one of drafting the account of its own conduct.

Publisher policy also requires operational disclosure, which we give here rather than in an acknowledgement. *Tool*: Claude (Anthropic), the same model in two harnesses — an agentic coding environment with shell, file, and browser tools (the repository work and this manuscript), and a design-native environment with live rendering (the revision of §9) — accessed 8–10 July 2026. *Manner of use*: the model wrote the code, the fonts’ construction data, the figures’ generators, and the prose of all three manuscripts, under the human author’s direction and criticism as documented throughout. *Human verification*: the author reviewed every rendered artifact and every revision round, supplied or relayed the external reviews, and accepted or rejected each result; the paper’s quantitative claims are recomputed from the archived record by `tools/paper_audit.py`, runnable by the author or any reader; every reference was checked against its source or publisher record, and the corrections that check forced are in the repository history. *Reason for use*: the AI-mediated production process is itself the object of study. The remaining sections of this paper are the long-form disclosure of what the tool did: Sections 4–6 in particular.

The configuration does expose a seam in the policy framework, which we note without proposing to resolve. Accountability-based authorship assumes the accountable party can, in principle, vouch for the work’s details. The human author here can vouch for the artifacts (they are testable), for the audit trail (it is complete), and for every judgment reported — but not, in the ordinary sense, for having written the sentences. The framework’s available answer is that the human’s editorial control, verification, and acceptance constitute the vouching — and the disclosure above is intended to make that verification concrete rather than notional: the artifacts are testable, the numbers are recomputable, the references were checked. The disclosure is designed to satisfy accountability-based policies; whether it does is each target venue’s determination to make. It is also, visibly, a policy under strain — and the strain is this paper’s final observation, not its complaint.

11 Limitations

$N=1$, and a favorable one: a self-contained project, a single decisive critic, no coordination costs, and a domain (type) where artifacts render instantly and judgment is visual. The account is written by an interested party. The mitigations of Section 3 bound but do not eliminate the risk of self-serving narrative; in particular, the selection of which episodes to narrate is a judgment this paper’s author made about this paper’s author. The perceived-versus-measured gap of Becker et al. (2025) warns that participants misjudge these collaborations from inside. Our defense is that the paper’s claims are, wherever possible, about committed artifacts rather than felt productivity. The postscript adds a second builder but not a second critic, so its transfer evidence bears on the task, not on critics. The author’s statement of intent — that the solution-free channel was a deliberate probe — is retrospective testimony, recorded after the events; it reframes the case but cannot be verified against the transcript, and the task itself was purposively chosen, a domain nearly ideal for artifact-level judgment. Finally, the critic’s effectiveness is confounded with the critic’s taste being *right*. A blunt critic with bad taste would compress the same loop toward worse artifacts, and nothing here measures that risk.

12 Conclusion

Two papers and their typeface were built by a machine that constructed rapidly but repeatedly failed to detect important defects in its own output, steered by a human who constructed nothing and evaluated everything — 401 words in the counted criticism channel — and who, by his own account, was running that asymmetry as an experiment: testing whether the machine could do skilled work he could not, and declining on method to point the way. What that criticism changed, on the record assembled here, was not primarily the artifacts but the builder’s method: patching gave way to preserving relationships, trusting compilation to testing behavior, deriving conventions to measuring exemplars, local repairs to reusable checks. This is one case — one critic, one task, one model family — and we offer it as

a documented configuration worth instrumenting, not as proof of a superior workflow. It comes with a portable toolkit (the ten rules of Section 8, every one a check an artifact can pass or fail) and an auditable trail for anyone who wants to reweigh it. The older lesson underneath is not new at all. The builder learned nothing from being right quickly, and everything from being told, precisely and without instructions, that it was wrong.

A The critique corpus

The complete critique channel of the first day’s afternoon loop, verbatim from the session transcript, with the single-coder post-hoc category assignment described in Section 3. An asterisk marks utterances with a flagged secondary function (e.g. a defect verdict that also points at sources). Rows coded *logistics*, *review*, and *infrastructure* are excluded from the critique counts. Seventeen screenshot-only messages interleave these utterances; their timestamps are in the corpus file. This table is generated from `data/critique_corpus.json` by the audit script and cannot drift from it.

#	Utterance (verbatim; typos preserved)	Category	Commit
1	figure 6 in the typography paper has curves that don’t meet properly, it’s pretty obvious, look and fix it please	defect	83dc417
2	the rendered chuzditsa font looks terrible, keep iterating until it looks professional	escalation	83dc417
3	are we done? is the latest of everything pushed?	logistics	–
4	what do you suggest as a response to this review of the two papers together? [pasted external review omitted]	review	4a96210
5	do it all, but also include the linguistic audit as supplemental / appendix on the language paper	review	4a96210
6	figure 7 in the typeface paper looks terrible, this is clearly not supporting its claim. fix it and make it look good. consult typesetting references if you need	defect*	9f6cac1
7	similarly, figure 10 shows obvious visual defects: the 3 is missing half of itself, the 4 has an empty square where its lines intersect, and the visual height of characters is generally inconsistent and jarring.	defect	593407c
8	also the kerning is absolutely terrible	defect	593407c
9	this is still obviously bad, keep improving	escalation	b63588b
10	the strokes still don’t align look at where the curves meet on the S shape - they don’t meet properly. check for mathematical and visual correctness	defect*	8002b5e
11	is the claim of the caption here true? it doesn’t seem to be	verification	1dd8d54
12	and also the numerals still looks terrible	defect	2f2451d
13	now figure 1 is borked	defect	00034eb
14	there is still a gap in kerning	defect	547af24
15	this is overfull now	defect	547af24
16	the character on the right has clearly misaligned strokes	defect	547af24
17	its getting worse. the skeleton of line elements don’t line up anymore. rethink your approach to modeling these things and make it sound	redirection	1ba1452
18	the glyphs are getting worse. do better	escalation	7977e4c
19	look at these examples visually, the errors are clear as day	defect	7977e4c
20	there are more mistakes, see this figure - the B shapes are clearly different when discussing stroke weight. the mathematical model is obviously incorrect, fix it	defect	bb360ee
21	the strokes on some of these numerals clearly don’t meet properly - the mathematical model is clearly incorrect, fix it	defect	2f2451d
22	this is not convincing, nobody would have confused those. this argument is not convincing	defect	9f6cac1
23	pdf doesn’t render again	infrastructure	–

#	Utterance (verbatim; typos preserved)	Category	Commit
24	it's still wrong. the vertical strokes clearly are off by a bit because you can see their edge where they meet the curve	defect	b305f0d
25	this is even worse than before. consult online open typefaces to learn how to do this properly	redirection	b305f0d
26	the kerning here is awful, consult online open typefaces - especially cyrillic ones - to learn how to kerning properly with this sort of typeface	defect*	f77ef68
27	all of these are offset. look online at typesetting manuals and guides to figure out how to do this properly	defect*	b305f0d
28	this is a typeface paper that can't make a typeface. look at online guides about how to make typefaces to figure out what you're doing wrong	redirection	b305f0d
29	now look at existing fonts to learn how to kern	redirection	f77ef68

References

- Becker, J., Rush, N., Barnes, E., & Rein, D. (2025). Measuring the impact of early-2025 AI on experienced open-source developer productivity. arXiv:2507.09089.
- COPE Council (2023). *Authorship and AI tools: COPE position statement*. Committee on Publication Ethics. <https://publicationethics.org/cope-position-statements/ai-author>
- Amershi, S., Cakmak, M., Knox, W. B., & Kulesza, T. (2014). Power to the people: The role of humans in interactive machine learning. *AI Magazine*, 35(4), 105–120.
- Ellis, C., Adams, T. E., & Bochner, A. P. (2011). Autoethnography: An overview. *Forum Qualitative Sozialforschung / Forum: Qualitative Social Research*, 12(1), Art. 10.
- Hofstadter, D. R. (1982). Metafont, metamathematics, and metaphysics: Comments on Donald Knuth's article "The Concept of a Meta-Font." *Visible Language*, 16(4), 309–338.
- Horvitz, E. (1999). Principles of mixed-initiative user interfaces. In *Proceedings of CHI 1999* (pp. 159–166). ACM.
- ICMJE (2025). *Recommendations for the conduct, reporting, editing, and publication of scholarly work in medical journals*. International Committee of Medical Journal Editors. (AI provisions first added May 2023.) <https://www.icmje.org/recommendations/>
- Karpathy, A. (2025). Post of 2 February 2025 introducing "vibe coding." X (formerly Twitter). Term subsequently lexicalized (Collins Dictionary Word of the Year, 2025).
- Knuth, D. E. (1982). The concept of a meta-font. *Visible Language*, 16(1), 3–27.
- Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The AI Scientist: Towards fully automated open-ended scientific discovery. arXiv:2408.06292.
- Nature (2023). Tools such as ChatGPT threaten transparent science; here are our ground rules for their use. *Nature*, 613, 612.
- Parasuraman, R., & Riley, V. (1997). Humans and automation: Use, misuse, disuse, abuse. *Human Factors*, 39(2), 230–253.
- Peng, S., Kalliamvakou, E., Cihon, P., & Demirer, M. (2023). The impact of AI on developer productivity: Evidence from GitHub Copilot. arXiv:2302.06590.
- Schmidgall, S., et al. (2025). Agent Laboratory: Using LLM agents as research assistants. arXiv:2501.04227.
- Schön, D. A. (1983). *The reflective practitioner: How professionals think in action*. New York: Basic Books.

- Si, C., Yang, D., & Hashimoto, T. (2024). Can LLMs generate novel research ideas? A large-scale human study with 100+ NLP researchers. arXiv:2409.04109.
- Thorp, H. H. (2023). ChatGPT is fun, but not an author. *Science*, 379(6630), 313.
- Uluoğlu, B. (2000). Design knowledge communicated in studio critiques. *Design Studies*, 21(1), 33–58.
- Yamada, Y., Lange, R. T., Lu, C., Hu, S., Lu, C., Foerster, J., Clune, J., & Ha, D. (2025). The AI Scientist-v2: Workshop-level automated scientific discovery via agentic tree search. arXiv:2504.08066.
- Zimmerman, J., Forlizzi, J., & Evenson, S. (2007). Research through design as a method for interaction design research in HCI. In *Proceedings of CHI 2007* (pp. 493–502). ACM.